



University
of Glasgow

COMPSCI5059 Software Engineering (IT) - 2025-26

Assessed Exercise 2 – Team Report

Team Name	Group 25	
Team Members	Makha Oyanontaruk	3162543o
	Yee Yuet Chow	3079077c
	Hannah Puff	3128814p
	Luke Roche	2474938r
	Simran Singh	3166877s
	Muhammad Abdullah bin Tariq	3059237t

1. Contribution of Team Members

Every group member contributed equally to this project.

Team Member Names	Contribution
Makha Oyanontaruk	Story Card #2 + Sequence Diagram TeachingRequirement class CommandLineInterface class
Yee Yuet Chow	Story Card #5 + Sequence Diagram TeacherManager class
Hannah Puff	Story Card #4 + Sequence Diagram Teacher class Main class
Luke Roche	Class Structure Diagram CommandLineInterface class Main class
Simran Singh	Story Card #3 + Sequence Diagram FileHandler class
Muhammad Abdullah bin Tariq	Story Card #1 + Sequence Diagram RequirementsManager class CommandLineInterface class

2. List of User Stories

The user stories are developed focusing on 3 user roles: 1) Class director, 2) PTT director, and 3) Administrator.

#1	Create Teaching Requirements List
	Description: As a Class Director, I want to create the list of teaching requirements, so that the administrator can take reference from or edit the list. Relevant User Roles: Class Director Priority: Must have Time Estimate: 0.5 workdays
	Acceptance Test: ☐ Class director can create a list of teaching requirements (e.g. course name, skills needed, hours) ☐ The list of teaching requirements is saved successfully after creating

#2	Edit Teaching Requirements List
	Description: As a PTT Director, I want to be able to edit the list of teaching requirements that the class director has created, so that I can keep the list accurate whenever there is a change in requirements or assigned teacher. Relevant User Roles: PTT Director Priority: Should have Time Estimate: 1.5 workdays
	Acceptance Test: ☐ PTT Director can access the created list of teaching requirements ☐ PTT Director can make edits on the list of teaching requirements ☐ The updated list of teaching requirements is saved successfully ☐ Teaching requirement details is correctly displayed

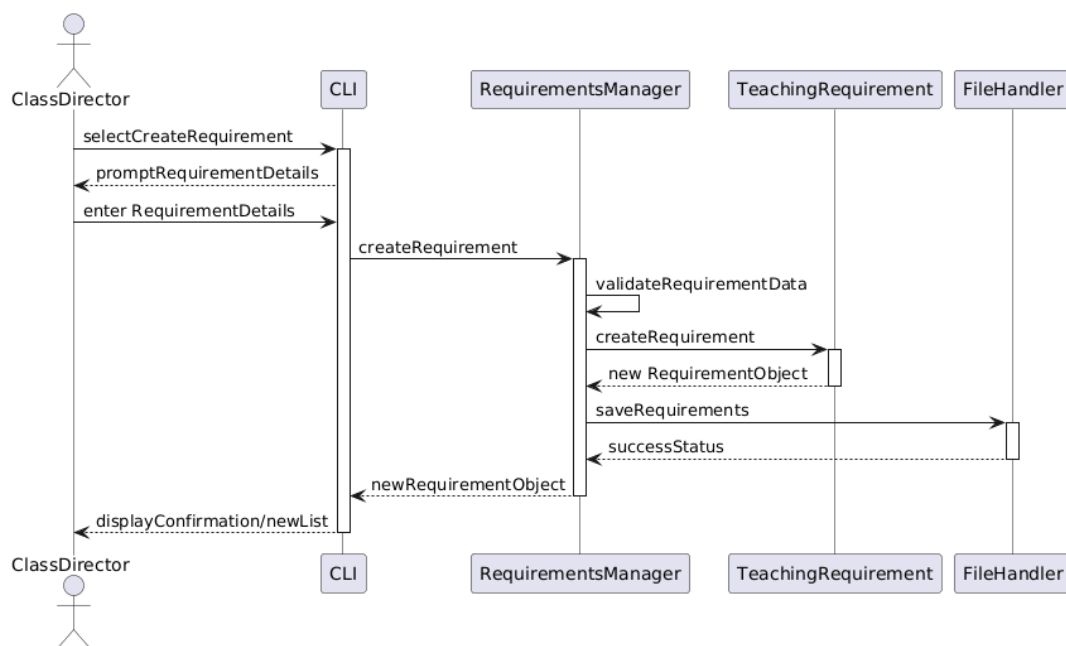
#3	Access/View Teaching Requirements List
	Description: As an administrator, I want to view the list of teaching requirements that the class director has created, so that I know what part-time staff I should be looking for to facilitate teaching. Relevant User Roles: Administrator Priority: Must have Time Estimate: 0.5 workdays
	Acceptance Test: ☐ Administrator can access the created list of teaching requirements ☐ Administrator can view the list of teaching requirements details (e.g. course name, skills needed, hours, assigned teacher) ☐ Teaching requirement details is correctly displayed

#4	Access/View Part Time Teachers Details
	Description: As an administrator, I want to be able to view part time teachers' profile, so that I can see if their profile/skills match the teaching requirements. Relevant User Roles: Administrator Priority: Must have Time Estimate: 0.5 workdays
	Acceptance Test: ☐ Administrator has access to the existing part time teacher profile list ☐ Administrator can view individual part time teacher profiles ☐ The profile information matches what was stored in the data

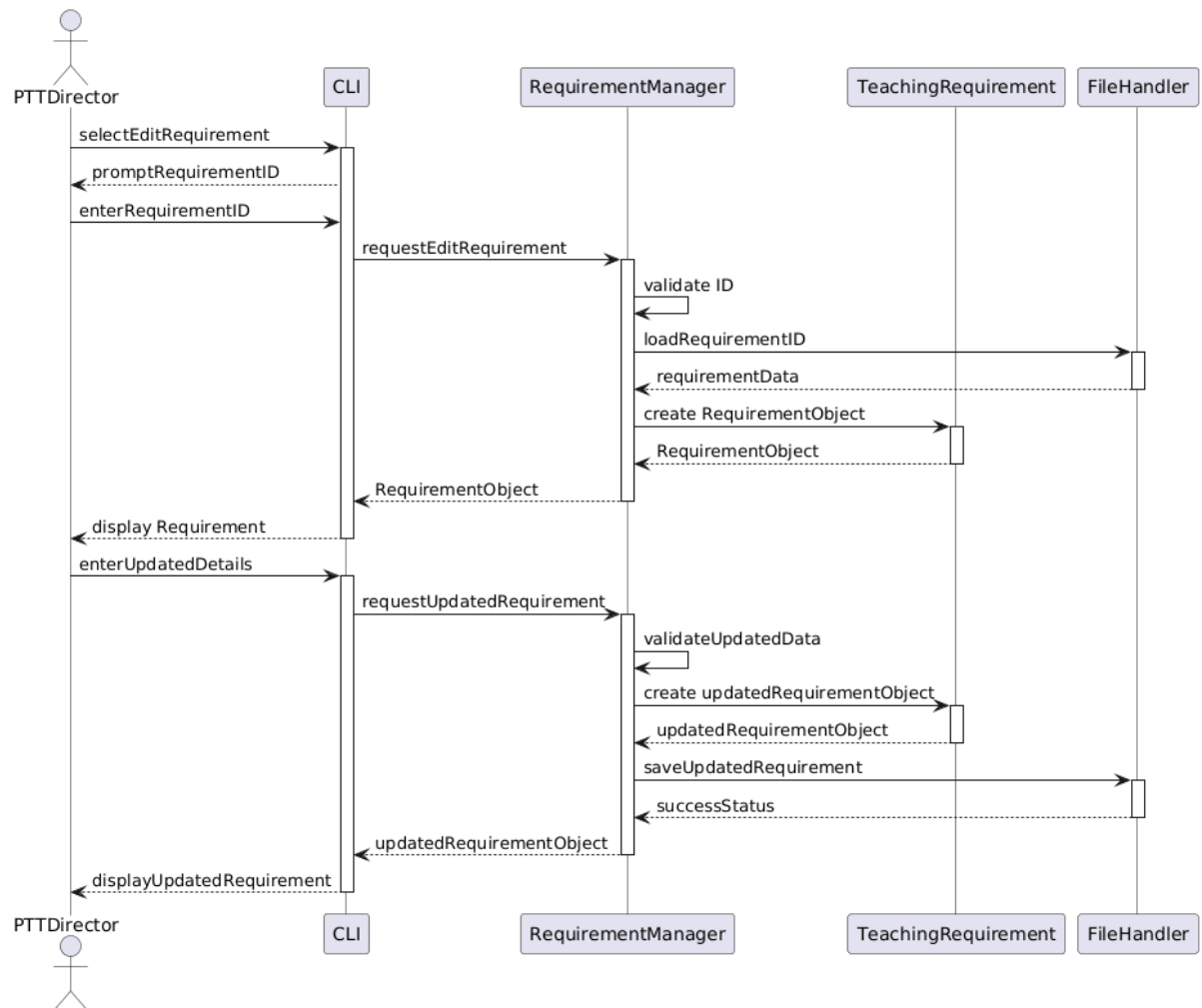
#5	Update Part Time Teachers Details
	Description: As an administrator, I want to be able to update the part time teacher's profile (e.g. if they have completed their training), so that I can organize training for the teachers effectively. Relevant User Roles: Administrator Priority: Should have Time Estimate: 1.5 workdays
	Acceptance Test: ☐ Administrator can edit the details in the part time teacher profiles ☐ The updated details in the profiles are saved successfully

3. UML Sequence Diagrams for User Stories

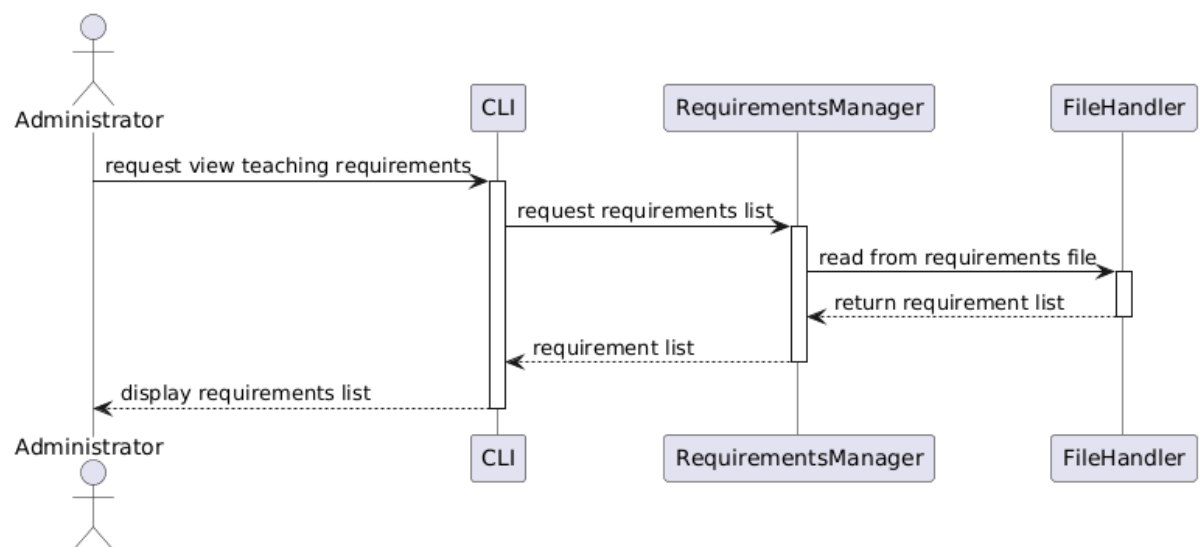
#1 Create Teaching Requirements List



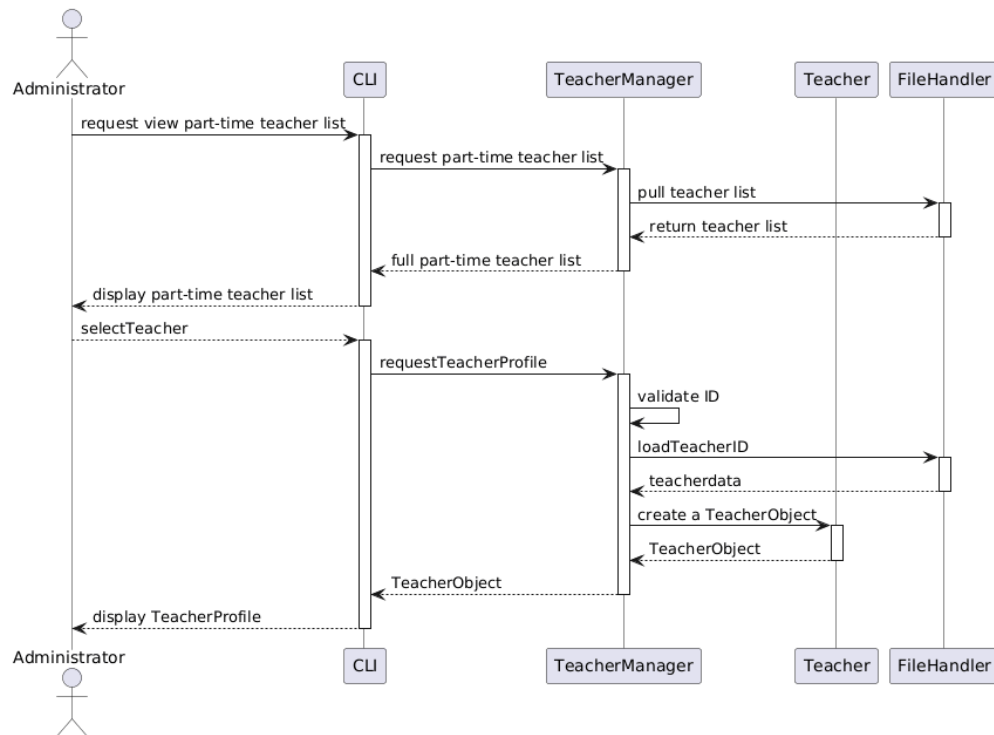
#2 Edit Teaching Requirements List



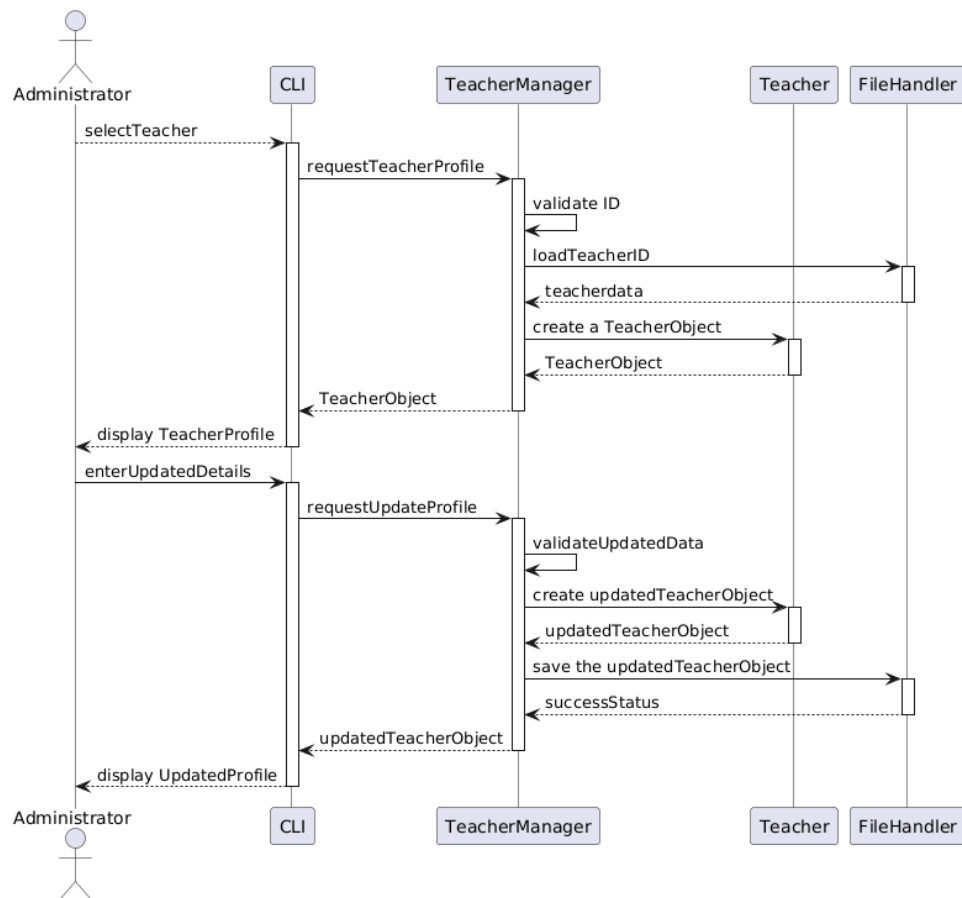
#3 Access/View Teaching Requirements List



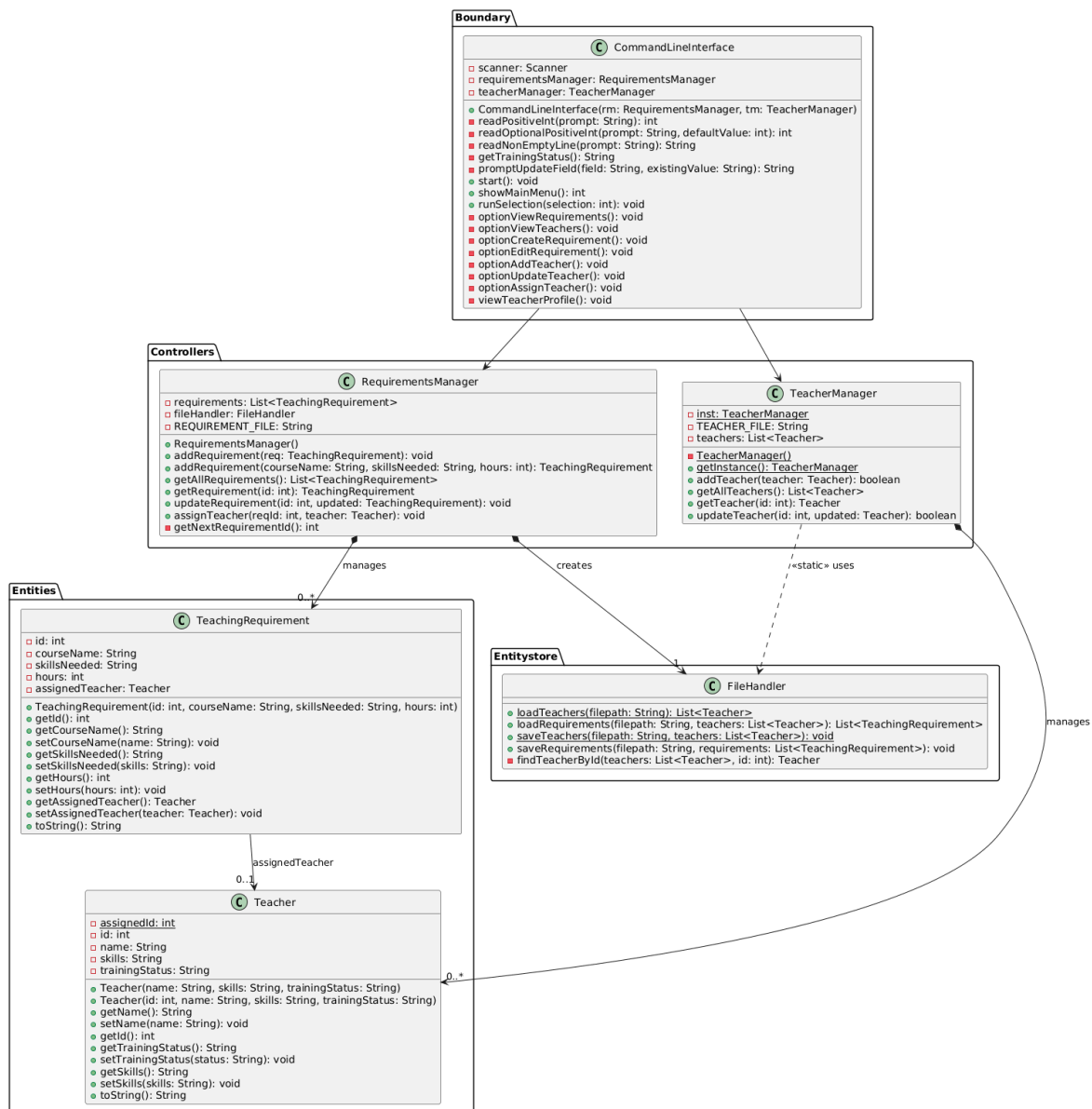
#4 Access/View Part Time Teachers Details



#5 Update Part Time Teachers Details



4. UML Class Structure Diagram



5. Design Commentary

Our group evaluated the system design on cohesion, coupling and access modifiers.

The Main class is responsible for starting the system and creating objects. It keeps object creation separate from object use. Main would be the only class that needs to be modified for future extensions.

The system consists of six core classes: **Teacher**, **TeacherManager**, **RequirementsManager**, **TeachingRequirement**, **FileHandler**, and **CommandLineInterface**. A Boundary-Control-Entity (BCE) architecture was adopted emphasising low coupling between each layer of architecture, high cohesion within the layers, and clear separation of responsibilities on user interactions, logic validation, class object creation or data storage.

The CommandLineInterface (CLI) class acts as the Boundary layer and handles all user interactions, including collecting user inputs, prompting and displaying returned outputs. CLI does not contain business logic – it is the main point of entry for users and passes commands to the Control classes.

The TeacherManager and RequirementManager classes serve as the Controls and coordinates the system logic and workflow. The TeacherManager follows the Singleton Design Pattern ensuring one instance and prevents duplicated or inconsistent data. Both TeacherManager and RequirementManager classes are the Façade. They provide unified and simplified interfaces like getTeacher() or updateRequirement() to the subsystems while performing ID validations, handle errors, loads and save data, and create entities behind the methods.

The Teacher and TeachingRequirement classes serve as the Entities and represents domain objects. These classes do not directly interact with the Control and Boundary layer. Teacher class holds name, skills, ID, and training status. The TeachingRequirements class holds course name, skills needed, and hours. This structuring ensures that changes to how data is processed would not require changes to the data structure itself. A Factory Design Pattern was adopted for entity creation. On evaluation, constructing objects directly in the CLI was found to reduce cohesion. Therefore, the entity creations are moved to the Control layer, specifically in the TeacherManager and RequirementManager classes.

The FileHandler class serves as an Entity Store in the system. It is responsible for handling all file operations including reading and writing files in the system. The FileHandler follows the Adaptor Design Pattern and acts as an adapter between the CSV files and the Control classes. This aims to avoid situations that the system becoming dependent on the file format and allows future migration to a new database with minimal impact to the system.

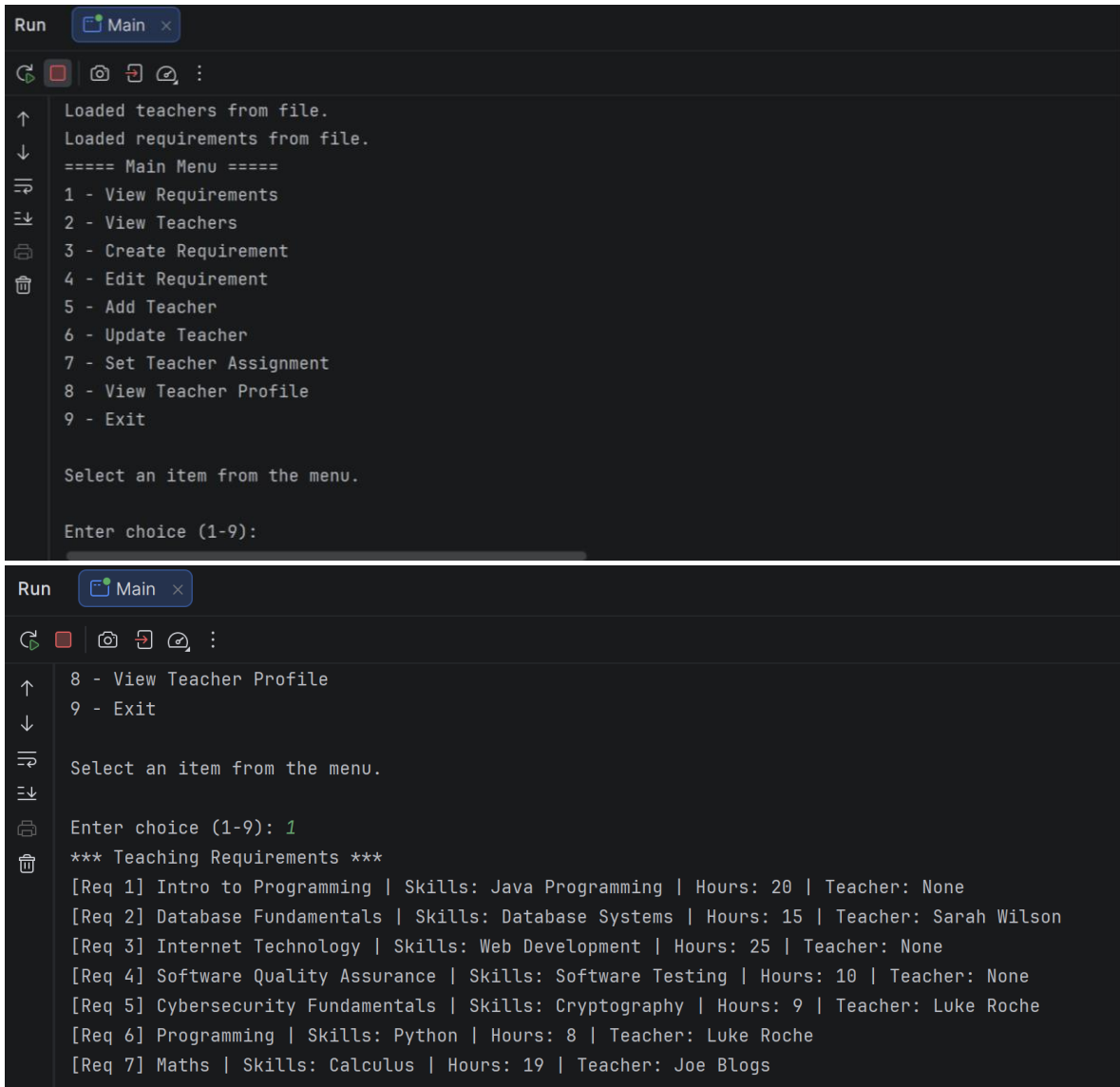
Additionally, strict encapsulation and close control on data access were implemented to reduce coupling. All entity fields are private and can only be accessed through the getters and setters' methods. This will prevent direct class manipulation of internal state and reduces content coupling.

Centralised of business logics was implemented within the Control layer, while also segregating the CLI, Teacher and TeachingRequirement classes from the FileHandler class. This approach reduced control coupling and avoided stamp coupling, which otherwise would have caused heavy dependence on file formats. This current architecture design ensures that the CSV file can easily be replaced with a database in future system versions. The CLI will just need to call the getter and setter methods and does not need to know how the data would be validated or loaded. These measures together ensured that functional cohesion was retained in the system.

Overall, clear separation responsibilities divided across classes improves cohesion, reduce coupling, and improve maintainability.

6. Implementation

(Screenshots illustrating our code functions as intended)



```
Run Main x
Loaded teachers from file.
Loaded requirements from file.
===== Main Menu =====
1 - View Requirements
2 - View Teachers
3 - Create Requirement
4 - Edit Requirement
5 - Add Teacher
6 - Update Teacher
7 - Set Teacher Assignment
8 - View Teacher Profile
9 - Exit

Select an item from the menu.

Enter choice (1-9):

Run Main x
8 - View Teacher Profile
9 - Exit

Select an item from the menu.

Enter choice (1-9): 1
*** Teaching Requirements ***
[Req 1] Intro to Programming | Skills: Java Programming | Hours: 20 | Teacher: None
[Req 2] Database Fundamentals | Skills: Database Systems | Hours: 15 | Teacher: Sarah Wilson
[Req 3] Internet Technology | Skills: Web Development | Hours: 25 | Teacher: None
[Req 4] Software Quality Assurance | Skills: Software Testing | Hours: 10 | Teacher: None
[Req 5] Cybersecurity Fundamentals | Skills: Cryptography | Hours: 9 | Teacher: Luke Roche
[Req 6] Programming | Skills: Python | Hours: 8 | Teacher: Luke Roche
[Req 7] Maths | Skills: Calculus | Hours: 19 | Teacher: Joe Blogs
===== Main Menu =====
```

```
Run Main x
Select an item from the menu.

Enter choice (1-9): 2
*** Teachers ***
[ID 1] | Name: John Smith | Skills: Java Programming | Training Status: Completed
[ID 2] | Name: Sarah Wilson | Skills: Database Systems | Training Status: In Progress
[ID 3] | Name: David Brown | Skills: Web Development | Training Status: Not Started
[ID 4] | Name: Emily Clark | Skills: Software Testing | Training Status: Completed
[ID 5] | Name: James Taylor | Skills: Machine Learning | Training Status: In Progress
[ID 6] | Name: Luke Roche | Skills: Python | Training Status: Completed
[ID 7] | Name: Joe Blogs | Skills: Calculus | Training Status: Completed
```

```
Select an item from the menu.

Enter choice (1-9): 3
*** Create Requirement ***
Enter course name: Cyber Security
Enter skills needed: cyber security certification
Enter number of hours: 65
Saved 9 requirements to file.
```

```
Run Main x
Select an item from the menu.

Enter choice (1-9): 4
*** Edit Requirement ***
Enter requirement id to update: 2
Current requirement:
[Req 2] Database Fundamentals | Skills: Database Systems | Hours: 15 | Teacher: Sarah Wilson
Leave a field blank to keep the current value.
New course name [Database Fundamentals]: Database Core Fundamentals
New skills needed [Database Systems]: Database SQL
New hours [15]: 20
Saved 9 requirements to file.
```

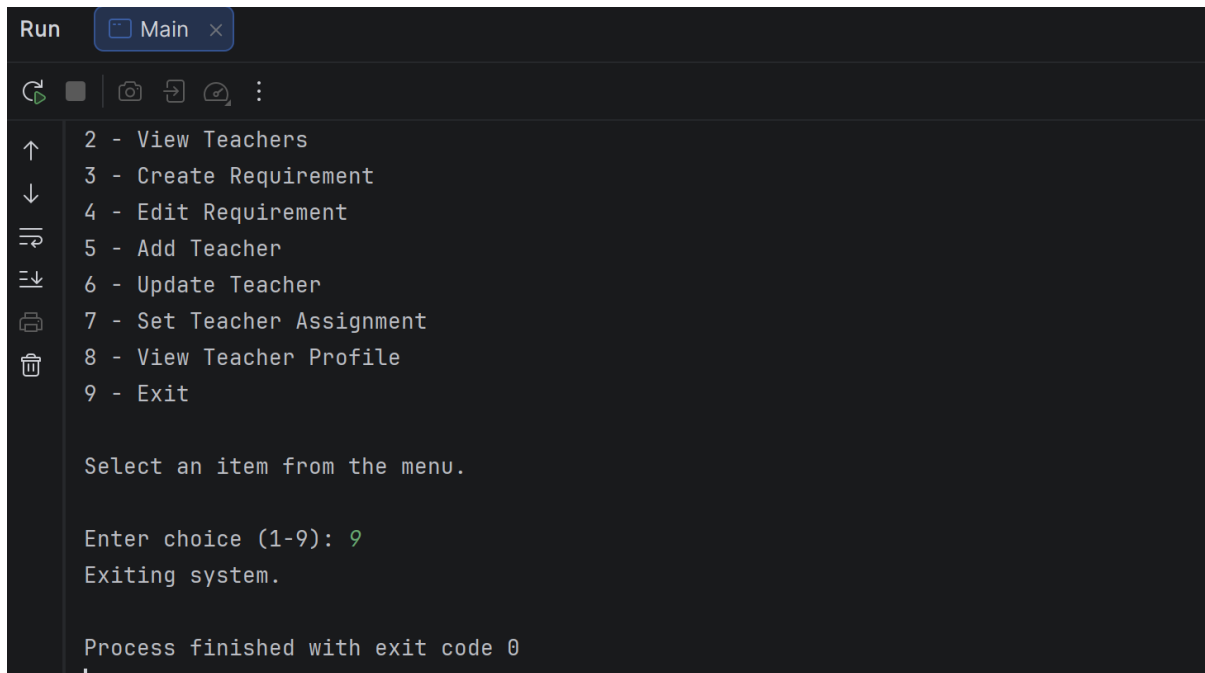
```
Run Main x
Select an item from the menu.

Enter choice (1-9): 5
*** Add Teacher ***
Enter teacher name: Professor Abs
Enter teacher skills: python coding
Enter training status:
1 - Completed
2 - In Progress
3 - Not Started
4 - Unknown
Enter choice (1-4): 2
Saved 8 teachers to file.
```

```
Run Main x
Enter choice (1-9): 6
*** Update Teacher ***
Enter the ID of the teacher profile you would like to UPDATE: 4
Current teacher profile:
[ID 4] | Name: Emily Clark | Skills: Software Testing | Training Status: Completed
Leave a field blank to keep the current value.
New name [Emily Clark]: Maggie Best
New skills [Software Testing]: Software in the workforce
New training status [Completed]: Enter training status:
1 - Completed
2 - In Progress
3 - Not Started
4 - Unknown
Enter choice (1-4): 3
Saved 8 teachers to file.
```

```
Run Main x
Enter choice (1-9): 7
*** Assign Teacher to Requirement ***
Enter requirement ID: 6
Enter teacher ID: 6
Saved 9 requirements to file.
Teacher assigned successfully:
[Req 6] Programming | Skills: Python | Hours: 8 | Teacher: Luke Roche
```

```
Run Main x
5 - Add Teacher
6 - Update Teacher
7 - Set Teacher Assignment
8 - View Teacher Profile
9 - Exit
Select an item from the menu.
Enter choice (1-9): 8
*** View Teacher Profile ***
Enter the ID of the teacher profile you would like to view: 3
[ID 3] | Name: David Brown | Skills: Web Development | Training Status: Not Started
```



```
Run Main x
2 - View Teachers
3 - Create Requirement
4 - Edit Requirement
5 - Add Teacher
6 - Update Teacher
7 - Set Teacher Assignment
8 - View Teacher Profile
9 - Exit

Select an item from the menu.

Enter choice (1-9): 9
Exiting system.

Process finished with exit code 0
```

7. Retrospective

Each member was assigned a user story to create the UMLs and code relating to that user story. The team was able to have productive group discussions and collaborate efficiently to complete the assignment.

What went well:

Everyone was available for group meetings which made division of work easy. Assigning the roles within the team kept things organized to avoid duplication of work. A share OneDrive folder ensured that all members of the team had access to the same documents and were able to see completion of task in real time. Communication happened frequently to prevent miscommunication and provide support if clarification was needed.

The team used GitHub for version control. Each member created their own branch for coding their assigned part. Regular communication was beneficial to avoid any merge conflict and to ensure everyone stayed informed. At least one team member was available to provide support should another member required assistance.

Aligning on the class structure and method names streamlined the development process as each team member had a clear understanding of the expected implementation. For example, the FileHandler was designed to use methods on the Teacher and TeachingRequirement prior to their implementation as method names and return types were already established.

Challenges:

The team encountered difficulties defining the systems boundaries, particularly with determining what should be included within the system and how the codebase should be structured in terms of the file organisation.

One of the main challenges was ensuring all modules integrated correctly. Even the granular errors in syntax, method names, incorrect return type or missing lines of logic cascaded into merge conflicts. Though effective team communications and clear separation of labour ensured that these issues were handled adequately.

The structure and organisation of the package had to be revised multiple times during the implementation phase. The project was augmented from two packages, initially, to four packages— in congruence with the system design. These changes required updating package names and imports across all files, which caused some refactoring issues. However, an active communication channel ensured these changes/issues were conveyed, acknowledged and addressed appropriately.

Learnings for the future:

Future improvements would be to set up the project on GitHub from the outset with the folder structure and empty class files already defined. This would ensure the team having a consistent base. Regular code merging and code review should be prioritised.

Overall, the team was able to collaborate to resolve any challenges encountered. Open communication and clearly defined roles significantly contributed to the success of the assignment.

8. Appendix

(Full codes here)

a) Sequence Diagram PlantUML codes

#1

```
1 @startuml
2 <style>
3   lifeLine {
4    LineStyle 0
5   }
6 </style>
7
8 actor ClassDirector
9 participant CLI
10 participant RequirementsManager as RM
11 participant TeachingRequirement as Req
12 participant FileHandler as R
13
14 ClassDirector -> CLI : selectCreateRequirement
15 activate CLI
16
17 CLI --> ClassDirector : promptRequirementDetails
18
19 ClassDirector -> CLI: enter RequirementDetails
20
21 CLI -> RM: createRequirement
22 activate RM
23
24 RM -> RM: validateRequirementData
25
26 RM -> Req: createRequirement
27 activate Req
28
29 Req --> RM: new RequirementObject
30 deactivate Req
31
32 RM -> R: saveRequirements
33 activate R
34
35 R --> RM: successStatus
36 deactivate R
37
38 RM --> CLI: newRequirementObject
39 deactivate RM
40
41 CLI --> ClassDirector: displayConfirmation/newList
42 deactivate CLI
43
44 @enduml
```

#2

```
1 @startuml
2
3 <style>
4 lifeline {
5    LineStyle 0
6 }
7 </style>
8
9 actor PTTDirector
10 participant CLI
11 participant RequirementManager as RM
12 participant TeachingRequirement as Req
13 participant FileHandler as R
14
15 PTTDirector -> CLI: selectEditRequirement
16 activate CLI
17
18 CLI --> PTTDirector: promptRequirementID
19
20 PTTDirector -> CLI: enterRequirementID
21
22 CLI -> RM: requestEditRequirement
23 activate RM
24
25 RM -> RM: validate ID
26
27 RM -> R: loadRequirementID
28 activate R
29
30 R --> RM: requirementData
31 deactivate R
32
33 RM -> Req: create RequirementObject
34 activate Req
35
36 Req --> RM: RequirementObject
37 deactivate Req
38
39 RM --> CLI: RequirementObject
40 deactivate RM
41
42 CLI --> PTTDirector: display Requirement
43 deactivate CLI
44
45 PTTDirector -> CLI: enterUpdatedDetails
46 activate CLI
47
48 CLI -> RM: requestUpdatedRequirement
49 activate RM
50
51 RM -> RM: validateUpdatedData
52
53 RM -> Req: create updatedRequirementObject
54 activate Req
55
56 Req --> RM: updatedRequirementObject
57 deactivate Req
58
59 RM -> R: saveUpdatedRequirement
60 activate R
61
62 R --> RM: successStatus
63 deactivate R
64
65 RM --> CLI: updatedRequirementObject
66 deactivate RM
67
68 CLI --> PTTDirector: displayUpdatedRequirement
69 deactivate CLI
70
71 @enduml
```

#3

```
1  @startuml
2
3  <style>
4  lifeline {
5      LineStyle 0
6  }
7  </style>
8
9  actor Administrator
10 participant "CLI" as CLI
11 participant "RequirementsManager" as RM
12 participant "FileHandler" as R
13
14 Administrator -> CLI : request view teaching requirements
15 activate CLI
16
17 CLI -> RM : request requirements list
18 activate RM
19
20 RM -> R : read from requirements file
21 activate R
22
23 R --> RM : return requirement list
24 deactivate R
25
26 RM --> CLI : requirement list
27 deactivate RM
28
29 CLI --> Administrator : display requirements list
30 deactivate
31
32 @enduml
```

#4

```
1 @startuml
2 <style>
3 lifeLine {
4    LineStyle 0
5 }
6 </style>
7 actor Administrator
8 participant CLI
9 participant TeacherManager
10 participant Teacher
11 participant FileHandler
12
13
14 Administrator -> CLI : request view part-time teacher list
15 activate CLI
16
17 CLI -> TeacherManager : request part-time teacher list
18 activate TeacherManager
19
20 TeacherManager -> FileHandler : pull teacher list
21 activate FileHandler
22
23 FileHandler --> TeacherManager : return teacher list
24 deactivate FileHandler
25
26
27 TeacherManager --> CLI: full part-time teacher list
28 deactivate TeacherManager
29
30 CLI --> Administrator: display part-time teacher list
31 deactivate CLI
32
33 Administrator --> CLI: selectTeacher
34 activate CLI
35
36 CLI-> TeacherManager: requestTeacherProfile
37 activate TeacherManager
38
39 TeacherManager -> TeacherManager: validate ID
40 TeacherManager -> FileHandler: loadTeacherID
41 activate FileHandler
42
43 FileHandler --> TeacherManager: teacherdata
44 deactivate FileHandler
45
46 TeacherManager -> Teacher: create a TeacherObject
47 activate Teacher
48
49 Teacher --> TeacherManager: TeacherObject
50 deactivate Teacher
51
52 TeacherManager --> CLI: TeacherObject
53 deactivate TeacherManager
54
55 CLI --> Administrator: display TeacherProfile
56 deactivate CLI
57
58 @enduml
```

#5

```
1 @startuml
2 <style>
3 lifeline {
4    LineStyle 0
5 }
6 </style>
7 actor Administrator
8
9 participant CLI
10 participant TeacherManager
11 participant Teacher
12 participant FileHandler
13
14 Administrator --> CLI: selectTeacher
15 activate CLI
16
17 CLI-> TeacherManager: requestTeacherProfile
18 activate TeacherManager
19
20 TeacherManager -> TeacherManager: validate ID
21 TeacherManager -> FileHandler: loadTeacherID
22 activate FileHandler
23
24 FileHandler --> TeacherManager: teacherdata
25 deactivate FileHandler
26
27 TeacherManager -> Teacher: create a TeacherObject
28 activate Teacher
29
30 Teacher --> TeacherManager: TeacherObject
31 deactivate Teacher
32
33 TeacherManager --> CLI: TeacherObject
34 deactivate TeacherManager
35
36 CLI --> Administrator: display TeacherProfile
37 deactivate CLI
38
39 Administrator -> CLI: enterUpdatedDetails
40 activate CLI
41
42 CLI -> TeacherManager: requestUpdateProfile
43 activate TeacherManager
44
45 TeacherManager -> TeacherManager: validateUpdatedData
46 TeacherManager-> Teacher: create updatedTeacherObject
47 activate Teacher
48
49 Teacher --> TeacherManager: updatedTeacherObject
50 deactivate Teacher
51
52 TeacherManager -> FileHandler: save the updatedTeacherObject
53 activate FileHandler
54
55 FileHandler --> TeacherManager: successStatus
56 deactivate FileHandler
57
58 TeacherManager --> CLI: updatedTeacherObject
59 deactivate TeacherManager
60
61 CLI --> Administrator: display UpdatedProfile
62 deactivate CLI
63
64 @enduml
```

b) *Codebase*

1. CODE: BOUNDARY--> **CommandLineInterface.Java**

```
package Boundary;
```

```
import java.util.List; import java.util.Scanner; import Entities.Teacher; import  
Entities.TeachingRequirement; import Controllers.RequirementsManager; import  
Controllers.TeacherManager;
```

```
/**
```

- Command-line interface for interacting with the teacher and requirement management system.
-
- Provides a menu-driven interface that allows the user to:
- -
 - View all teaching requirements or teacher profiles
 -
 - Create, edit, or delete requirements
 -
 - Add or update teacher profiles
 -
 - Assign a teacher to a requirement
 -
 - View a specific teacher's profile
 -
- Input validation is performed to ensure data integrity.

```
*/ public class CommandLineInterface { private final Scanner scanner; private final RequirementsManager  
requirementsManager; private final TeacherManager teacherManager;
```

```
/**
```

```
* Constructs a new CLI with the given managers.
```

```
*
```

```
* @param rm the manager for teaching requirements
```

```
* @param tm the manager for teacher data
```

```
*/
```

```
public CommandLineInterface(RequirementsManager rm, TeacherManager tm) {  
    this.scanner = new Scanner(System.in);  
    this.requirementsManager = rm;  
    this.teacherManager = tm;  
}
```

```
// ===== VALIDATION HELPERS =====
```

```
/**
```

```
* Prompts the user and reads a positive integer.
```

```
* Keeps asking until a valid positive number is entered.
```

```

*
* @param prompt the message to display to the user
* @return the positive integer entered
*/
private int readPositiveInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();
        try {
            int value = Integer.parseInt(input);
            if (value > 0) {
                return value;
            }
            System.out.println("Please enter a number greater than 0.");
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a whole number.");
        }
    }
}

```

```

/**
 * Prompts the user for an optional positive integer.
 * If the user enters nothing, the given default value is returned.
 *
 * @param prompt the message to display
 * @param defaultValue the value to return if input is empty
 * @return the positive integer entered, or the default
 */

```

```

private int readOptionalPositiveInt(String prompt, int defaultValue) {
    while (true) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();
        if (input.isEmpty()) {
            return defaultValue;
        }
        try {
            int value = Integer.parseInt(input);
            if (value > 0) {
                return value;
            }
            System.out.println("Please enter a number greater than 0.");
        }
        catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a whole number.");
        }
    }
}

```

```

/**
 * Prompts the user for a non-empty string.
 *
 * @param prompt the message to display
 * @return the trimmed input string (never empty)

```

```

*/
private String readNonEmptyLine(String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();
        if (!input.isEmpty()) {
            return input;
        }
        System.out.println("This field cannot be empty.");
    }
}

/**
 * Presents a menu of training status options and returns the user's choice.
 * If the user presses Enter with no input, an empty string is returned.
 *
 * @return the selected training status, or an empty string if no choice was made
 */
private String getTrainingStatus() {
    String[] trainingStatusOptions = new String[] { "Completed", "In Progress", "Not Started", "Unknown" };

    while (true) {
        System.out.println("Enter training status:");
        for (int i = 0; i < trainingStatusOptions.length; i++) {
            System.out.println((i+1) + " - " + trainingStatusOptions[i]);
        }

        System.out.print("Enter choice (1-" + trainingStatusOptions.length + "): ");

        String input = scanner.nextLine();
        if (input.trim().isEmpty()) {
            return "";
        }

        try {
            int choice = Integer.parseInt(input.trim());
            if (choice >= 1 && choice <= trainingStatusOptions.length) {
                return trainingStatusOptions[choice - 1];
            }
        } catch (NumberFormatException ignored) {}
        //int choice = Integer.parseInt(readNonEmptyLine("Enter choice (1-" + trainingStatusOptions.length + "):"));

        System.out.println("Invalid input. Please enter a number between 1 and " + trainingStatusOptions.length +
            ", or press Enter to leave blank.");
    }
}

/**
 * Prompts the user to update a specific field of a teacher.

```

```

* Shows the current value in brackets. For the "training status" field,
* the {@link #getTrainingStatus()} method is used; otherwise a simple text input.
* If the user enters nothing, the existing value is kept.
*
* @param field      the name of the field being updated (used in prompt)
* @param existingValue the current value of the field
* @return the new value (or the existing value if input was empty)
*/
private String promptUpdateField(String field, String existingValue) {

```

```

    System.out.print("New " + field + " [" + existingValue + "]: ");
    String update;
    if (field.equals("training status")) {
        update = getTrainingStatus();
    }
    else {
        update = scanner.nextLine().trim();
    }

```

```

    if (update.isEmpty()){
        update = existingValue;
    }

```

```

    return update;

```

```

}

```

```

// ===== MAIN MENU & LOOP =====

```

```

/**

```

```

* Starts the main command-line loop.
* Displays the menu, processes the user's choice, and repeats until exit.
*/

```

```

public void start() {
    while (true) {
        int choice = showMainMenu();
        if (choice == 9){
            System.out.println("Exiting system.");
            break;
        }
        runSelection(choice);
    }
}

```

```

/**

```

```

* Displays the main menu and reads the user's selection.
*
* @return a number between 1 and 9 (inclusive) representing the chosen option
*/

```

```

public int showMainMenu() {

```

```

String menu = ""
===== Main Menu =====
1 - View Requirements
2 - View Teachers
3 - Create Requirement
4 - Edit Requirement
5 - Add Teacher
6 - Update Teacher
7 - Set Teacher Assignment
8 - View Teacher Profile
9 - Exit

Select an item from the menu.
"";
final int NO_OF_OPTIONS = 9;

while (true) {
    System.out.println(menu);
    System.out.print("Enter choice (1-" + NO_OF_OPTIONS + "): ");
    String option = scanner.nextLine().trim();

    try {
        int selection = Integer.parseInt(option);
        if (selection >= 1 && selection <= NO_OF_OPTIONS) {
            return selection; // immediate return when valid
        } else {
            System.out.println("Please enter a number between 1 and " + NO_OF_OPTIONS + ".");
        }
    } catch (NumberFormatException e) {
        System.out.println("Invalid input. Please enter a number.");
    }
}

}

/**
 * Executes the action corresponding to the given menu selection.
 *
 * @param selection the menu option (1-8, as 9 is handled separately)
 */
public void runSelection(int selection) {
    Runnable[] actions = new Runnable[] {
        this::optionViewRequirements,
        this::optionViewTeachers,
        this::optionCreateRequirement,
        this::optionEditRequirement,
        this::optionAddTeacher,
        this::optionUpdateTeacher,
        this::optionAssignTeacher,
        this::viewTeacherProfile,
    };
};

```

```

        actions[selection-1].run();
    }

// ===== MENU ACTIONS =====

/** Displays all teaching requirements. */
private void optionViewRequirements() {
    List<TeachingRequirement> reqs = requirementsManager.getAllRequirements();
    if (reqs == null || reqs.isEmpty()){
        System.out.println("No teaching requirements found.");
        return;
    }
    System.out.println("*** Teaching Requirements ***");
    for (TeachingRequirement req : reqs) {
        System.out.println(req);
    }
}

/** Displays all teacher profiles. */
private void optionViewTeachers() {
    List<Teacher> teachers = teacherManager.getAllTeachers();

    if (teachers == null || teachers.isEmpty()){
        System.out.println("No teacher profiles found.");
        return;
    }
    System.out.println("*** Teachers ***");
    for (Teacher teacher:teachers){
        System.out.println(teacher);
    }
}

/** Guides the user through creating a new teaching requirement. */
private void optionCreateRequirement() {
    System.out.println("*** Create Requirement ***");
    String courseName = readNonEmptyLine("Enter course name: ");
    String skillsNeeded = readNonEmptyLine("Enter skills needed: ");
    int hours = readPositiveInt("Enter number of hours: ");

    TeachingRequirement req = requirementsManager.addRequirement(courseName, skillsNeeded, hours);
    if (req == null){
        System.out.println("Failed to create requirement.");
    } else {
        System.out.println("Requirement created successfully.");
        System.out.println(req);
    }
}

/** Allows the user to edit an existing teaching requirement. */

```

```

private void optionEditRequirement() {
    System.out.println("*** Edit Requirement ***");
    int id = readPositiveInt("Enter requirement id to update: ");
    TeachingRequirement existing = requirementsManager.getRequirement(id);
    if (existing == null) {
        System.out.println("Requirement not found.");
        return;
    }
    System.out.println("Current requirement:");
    System.out.println(existing);
    System.out.println("Leave a field blank to keep the current value.");

    String courseName = promptUpdateField("course name", existing.getCourseName());
    String skillsNeeded = promptUpdateField("skills needed", existing.getSkillsNeeded());
    int hours = readOptionalPositiveInt("New hours [" + existing.getHours() + "]: ", existing.getHours());

    TeachingRequirement updated = new TeachingRequirement(id, courseName, skillsNeeded, hours);
    requirementsManager.updateRequirement(id, updated);
    System.out.println("Requirement updated successfully.");
    System.out.println(requirementsManager.getRequirement(id));
}

/** Guides the user through adding a new teacher. */
private void optionAddTeacher() {
    // (Done) Incomplete code here, just put stuff that TeacherManager needs
    // Edit: Added the logic to add new teacher
    System.out.println("*** Add Teacher ***");
    String name = readNonEmptyLine("Enter teacher name: ");
    String skills = readNonEmptyLine("Enter teacher skills: ");
    String trainingStatus;

    while (true) {
        trainingStatus = getTrainingStatus();
        if (trainingStatus.isEmpty()) {
            System.out.println("You must enter the teacher's training status.");
            continue;
        }
        break;
    }

    Teacher newTeacher = new Teacher(name, skills, trainingStatus);
    boolean success = teacherManager.addTeacher(newTeacher);
    if (!success) {
        System.out.println("Teacher ID already exists in the system. Failed to add teacher.");
    }
    else {
        // Success message feedback to user
        System.out.println("New teacher added successfully.");
    }
}

```

```

        System.out.println(newTeacher); //just for clarity
    }
}

/** Allows the user to update an existing teacher's details. */
private void optionUpdateTeacher(){
    System.out.println("*** Update Teacher ***");
    // System.out.println("Enter the ID of the teacher profile you would like to UPDATE: ");
    int id = readPositiveInt("Enter the ID of the teacher profile you would like to UPDATE: ");
    // (Done) This is incomplete, just put logics that TeacherManager class needs (esp the booleans)
    //Edit: Added logics fro name, skills, training status
    Teacher existingTeacher = teacherManager.getTeacher(id);
    if (existingTeacher == null) {
        System.out.println("Teacher profile not found. Failed to update teacher profile.");
        return;
    }
    System.out.println("Current teacher profile:");
    System.out.println(existingTeacher);
    System.out.println("Leave a field blank to keep the current value.");

    String name = promptUpdateField("name", existingTeacher.getName());
    String skills = promptUpdateField("skills", existingTeacher.getSkills());
    String trainingStatus = promptUpdateField("training status", existingTeacher.getTrainingStatus());

    Teacher updatedTeacher = new Teacher(id, name, skills, trainingStatus);
    boolean success = teacherManager.updateTeacher(id, updatedTeacher);
    if (success) {
        System.out.println("Teacher profile updated successfully.");
        System.out.println(teacherManager.getTeacher(id));
    }
    else {
        System.out.println("Teacher profile not found. Failed to update teacher profile.");
    }
}

/** Assigns a teacher to a teaching requirement. */
private void optionAssignTeacher() {
    System.out.println("*** Assign Teacher to Requirement ***");
    int reqId= readPositiveInt("Enter requirement ID: ");
    TeachingRequirement requirement = requirementsManager.getRequirement(reqId);
    if (requirement ==null){
        System.out.println("Requirement not found.");
        return;
    }
    int teacherId = readPositiveInt("Enter teacher ID: ");
    Teacher teacher = teacherManager.getTeacher(teacherId);

    if (teacher ==null){
        System.out.println("Teacher not found.");
        return;
    }
}

```

```

        requirementsManager.assignTeacher(reqId, teacher);
        System.out.println("Teacher assigned successfully:");
        System.out.println(requirementsManager.getRequirement(reqId));
    }

    /** Displays the full profile of a single teacher. */
    private void viewTeacherProfile() {
        System.out.println("*** View Teacher Profile ***");

        int id = readPositiveInt("Enter the ID of the teacher profile you would like to view: ");
        Teacher teacher = teacherManager.getTeacher(id);

        if (teacher == null) {
            System.out.println("Teacher not found.");
        }
        else {
            System.out.println(teacher);
        }
    }
}

```

2. CODE: CONTROLLERS FOLDER RequirementManager. Java

```

package Controllers;

import java.util.ArrayList; import java.util.List;

import Entities.Teacher; import Entities.TeachingRequirement; import Entitystore.FileHandler;

/**

```

- Manages teaching requirements in the system.
-
- This controller handles:
- -
 - Loading requirements from persistent storage
 -
 - Adding new requirements (with validation)
 -
 - Retrieving individual or all requirements
 -
 - Updating existing requirement details
 -
 - Assigning a teacher to a requirement
 -
 - Saving changes back to storage
 -

-
- It relies on the {@link TeachingRequirement} class providing the necessary
- getters and setters (getId, getCourseName, getSkillsNeeded, getHours,
- setCourseName, setSkillsNeeded, setHours, setAssignedTeacher).

```
*/ public class RequirementsManager { private List requirements; private final FileHandler fileHandler; private
static final String REQUIREMENT_FILE = "data/requirements.csv";
```

```
/**
```

```
* Constructs a RequirementsManager and loads existing requirements from the CSV file.
* Teacher data is obtained from the {@link TeacherManager} singleton to properly
* re-link assigned teachers when loading.
```

```
*/public RequirementsManager() { fileHandler = new FileHandler();
    TeacherManager teacherManager = TeacherManager.getInstance();
    List<Teacher> teachers = teacherManager.getAllTeachers();
    requirements = fileHandler.loadRequirements(REQUIREMENT_FILE, teachers);
    if (requirements == null){
        requirements = new ArrayList<>();
    }
}
```

```
/**
```

```
* Adds a new teaching requirement to the system.
* The requirement is validated (non-null, non-blank course name and skills,
* positive hours) and checked for duplicate ID before adding.
* If successful, the updated list is saved to the CSV file.
```

```
*
* @param req the TeachingRequirement object to add
*/
```

```
public void addRequirement(TeachingRequirement req) {
    if (req == null){
        return;
    }
    if (req.getCourseName() == null || req.getCourseName().trim().isEmpty()) {
        return;
    }
    if (req.getSkillsNeeded() == null || req.getSkillsNeeded().trim().isEmpty()) {
        return;
    }
    if (req.getHours() <= 0) {
        return;
    }
    if (getRequirement(req.getId()) == null){
        requirements.add(req);
        fileHandler.saveRequirements(REQUIREMENT_FILE, requirements);
    }
}
```

```
/**
```

```
* Creates and adds a new teaching requirement from individual fields.
* Validates input, generates a new unique ID, creates a TeachingRequirement object,
```

```

* and delegates to {@link #addRequirement(TeachingRequirement)}.
*
* @param courseName the name of the course
* @param skillsNeeded the required skills (as a descriptive string)
* @param hours the total teaching hours required (must be positive)
* @return the newly created TeachingRequirement object, or {@code null} if validation fails
*/
public TeachingRequirement addRequirement(String courseName, String skillsNeeded, int hours) {
    if (courseName == null || courseName.trim().isEmpty()) {
        return null;
    }
    if (skillsNeeded == null || skillsNeeded.trim().isEmpty()) {
        return null;
    }
    if (hours <= 0) {
        return null;
    }
    int newId = getNextRequirementId();
    TeachingRequirement req = new TeachingRequirement(newId, courseName, skillsNeeded, hours);
    addRequirement(req);
    return req;
}

/**
 * Returns a copy of the list of all teaching requirements.
 *
 * @return a new ArrayList containing all requirements
 */
public List<TeachingRequirement> getAllRequirements() {
    return new ArrayList<>(requirements);
}

/**
 * Retrieves a teaching requirement by its unique ID.
 *
 * @param id the requirement ID to search for
 * @return the TeachingRequirement with the matching ID, or {@code null} if not found
 */
public TeachingRequirement getRequirement(int id) {
    for (TeachingRequirement req : requirements) {
        if (req.getId() == id) {
            return req;
        }
    }
    return null;
}

/**
 * Updates an existing teaching requirement with new details.
 * The requirement to update is identified by its ID. The new values are taken
 * from the {@code updated} object after validation. If the update is successful,

```

```

* the full list is saved to the CSV file.
*
* @param id    the ID of the requirement to update
* @param updated a TeachingRequirement object containing the new values
*/
public void updateRequirement(int id, TeachingRequirement updated){
    if (updated == null){
        return;
    }
    if (updated.getCourseName() ==null || updated.getCourseName().trim().isEmpty()){
        return;
    }
    if (updated.getSkillsNeeded() ==null || updated.getSkillsNeeded().trim().isEmpty()){
        return;
    }
    if (updated.getHours() <= 0){
        return;
    }
    TeachingRequirement existing = getRequirement(id);
    if (existing != null){
        existing.setCourseName(updated.getCourseName());
        existing.setSkillsNeeded(updated.getSkillsNeeded());
        existing.setHours(updated.getHours());
        fileHandler.saveRequirements(REQUIREMENT_FILE, requirements);
    }
}

```

```

/**
* Assigns a teacher to a teaching requirement.
* Both the requirement (by ID) and the teacher must exist (non-null).
* After assignment, the updated list is saved to the CSV file.
*
* @param reqId  the ID of the requirement
* @param teacher the Teacher to assign (must not be {@code null})
*/
public void assignTeacher(int reqId, Teacher teacher){
    if (teacher == null){
        return;
    }
    TeachingRequirement requirement = getRequirement(reqId);
    if(requirement!=null){
        requirement.setAssignedTeacher(teacher);
        fileHandler.saveRequirements(REQUIREMENT_FILE, requirements);
    }
}

```

```

/**
* Calculates the next available ID for a new requirement by finding the current
* maximum ID and incrementing it.
*

```

```

    * @return the next unused requirement ID
    */
private int getNextRequirementId(){
    int maxId =0;
    for(TeachingRequirement req :requirements){
        if(req.getId() >maxId){
            maxId= req.getId();
        }
    }
    return maxId +1;
}
}

```

3. CODE: CONTROLLERS FOLDER, TeacherManager.java

```
package Controllers;
```

```
import Entitystore.FileHandler; import Entities.Teacher; import java.util.ArrayList; import java.util.List;
```

```
/**
```

- Manages all teacher-related operations in the system.
-
- This controller class handles:
- -
 - Loading teacher data from persistent storage
 -
 - Adding new teachers
 -
 - Retrieving individual or all teachers
 -
 - Updating existing teacher details
 -
 - Saving changes back to storage
 -
- It follows the Singleton design pattern to ensure a single point of control
- throughout the application.

```
*/ public class TeacherManager { // Singleton Pattern (only one instance + need global access point) private
static TeacherManager inst;
```

```
private static final String TEACHER_FILE = "data/teachers.csv";
private List<Teacher> teachers;
```

```
/**
```

```

    * Private constructor to prevent direct instantiation.
    * Loads existing teacher data from the CSV file; if loading fails,
    * initializes an empty list.
    */

```

```
private TeacherManager() {
    teachers = FileHandler.loadTeachers(TEACHER_FILE);
}
```

```

        if (teachers == null) {
            teachers = new ArrayList<>();
        }
    }

/**
 * Returns the singleton instance of TeacherManager.
 * Creates the instance on the first call.
 *
 * @return the single TeacherManager instance
 */
public static TeacherManager getInstance() {
    if (inst == null) { inst = new TeacherManager();
    }
    return inst;
}

/**
 * Adds a new teacher to the system.
 * Performs validation to prevent null teachers and duplicate IDs.
 * If successful, the updated teacher list is saved to the CSV file.
 *
 * @param teacher the Teacher object to add
 * @return true if the teacher was added successfully, false otherwise
 */
public boolean addTeacher(Teacher teacher) {
    if (teacher == null) {
        return false;
    }

    for (Teacher t : teachers) {
        if (t.getId() == teacher.getId()) {
            return false;
        }
    }
    teachers.add(teacher);

    FileHandler.saveTeachers(TEACHER_FILE, teachers);
    return true;
}

/**
 * Returns a list of all teachers currently in the system.
 * If the list is empty or null, a message is printed to the console.
 *
 * @return the list of teachers (may be empty)
 */
public List<Teacher> getAllTeachers() {
    if (teachers == null || teachers.isEmpty()) {
        System.out.println("No teacher profile is found in the system."); // Handles edge case
    }
    return teachers;
}

```

```

}

/**
 * Retrieves a teacher by their unique ID.
 * If no teacher with the given ID exists, a message is printed and null is returned.
 *
 * @param id the teacher ID to search for
 * @return the Teacher object with the matching ID, or null if not found
 */
public Teacher getTeacher(int id) {
    for (Teacher t : teachers) {
        if (t.getId() == id) {
            return t;
        }
    }
    System.out.println("Unable to find teacher with ID: " + id);
    return null;
}

/**
 * Updates the details of an existing teacher.
 * The teacher to update is identified by the provided ID.
 * Only the name, skills, and training status are updated from the
 * {@code updated} parameter. After updating, the entire teacher list
 * is saved to the CSV file.
 *
 * @param id the ID of the teacher to update
 * @param updated a Teacher object containing the new values
 * @return true if the teacher was found and updated, false otherwise
 */
public boolean updateTeacher(int id, Teacher updated) {
    for (int i=0; i < teachers.size(); i++) {
        if (teachers.get(i).getId() == id) {
            Teacher teacherToBeUpdated = teachers.get(i);
            teacherToBeUpdated.setName(updated.getName());
            teacherToBeUpdated.setSkills(updated.getSkills());
            teacherToBeUpdated.setTrainingStatus(updated.getTrainingStatus());
            FileHandler.saveTeachers(TEACHER_FILE, teachers);

            return true;
        }
    }
    return false;
}

}

```

4. CODE: ENTITIES--> Teacher.Java

```
package Entities;
```

```
/**
```

- Represents a teacher with name, skills, and training status.
- Each teacher is assigned a unique ID automatically upon creation.

```
*/ public class Teacher { private static int assignedId = 1; private final int id; private String name; private String skills; private String trainingStatus;
```

```
/**
```

```
 * Constructs a new Teacher with an auto-generated ID.
```

```
 *
```

```
 * @param name      the teacher's name
```

```
 * @param skills    the teacher's skills
```

```
 * @param trainingStatus the teacher's current training status e.g. Completed, In Progress.
```

```
 */
```

```
public Teacher(String name, String skills, String trainingStatus){  
    this.id = assignedId++;  
    this.name = name;  
    this.trainingStatus = trainingStatus;  
    this.skills = skills;  
}
```

```
/**
```

```
 * Constructs a Teacher with a predefined ID (e.g. when loading from a file).
```

```
 * Ensures that the static ID counter does not fall behind existing IDs.
```

```
 *
```

```
 * @param id        the teacher's existing ID
```

```
 * @param name      the teacher's name
```

```
 * @param skills    the teacher's skills
```

```
 * @param trainingStatus the teacher's training status
```

```
 */
```

```
public Teacher(int id, String name, String skills, String trainingStatus){  
    this.id = id;  
    if (id >= assignedId) {  
        assignedId = id + 1;  
    }  
    this.name = name;  
    this.skills = skills;  
    this.trainingStatus = trainingStatus;
```

```
}
```

```
/**
```

```
 * getName() returns name
```

```
 * setName(String name) sets name
```

```
 * getSkills() returns skills
```

```
 * setSkills(String skills) sets skills
```

```
 * getTrainingStatus() returns training status
```

```
 * setTrainingStatus(String status) sets training status
```

```
 * getId() returns the teacher's id
```

```
 * toString() returns a readable summary
```

```

    * all my initialized parameters name, skills, training_status, and id
    */
    public String getName(){return name;}
    public void setName(String name){this.name=name;}
    public int getId(){return id;}
    public String getTrainingStatus(){return trainingStatus;}
    public void setTrainingStatus(String status){this.trainingStatus= status;}
    public String getSkills(){return skills;}
    public void setSkills(String skills){this.skills=skills;}

    @Override
    public String toString(){
        return String.format("[ID %s] | Name: %s | Skills: %s | Training Status: %s",id,name,skills,trainingStatus);
    }

}

```

5. CODE: ENTITIES--> TeacherManager.Java

```
package Entities;
```

```
/**
```

- Represents a teaching requirement, including course name, required skills,
- total hours, and an optionally assigned teacher.

```
*/ public class TeachingRequirement { private final int id; private String courseName; private String
skillsNeeded; private int hours; private Teacher assignedTeacher;
```

```
/**
```

```
* Constructs a teaching requirement with the given details.
```

```
* Initially no teacher is assigned.
```

```
*
```

```
* @param id the unique identifier for this requirement
```

```
* @param courseName the name of the course
```

```
* @param skillsNeeded description of skills required to teach the course
```

```
* @param hours the total teaching hours required
```

```
*/
```

```
public TeachingRequirement(int id, String courseName, String skillsNeeded, int hours) {
    this.id = id;
    this.courseName = courseName;
    this.skillsNeeded = skillsNeeded;
    this.hours = hours;
    this.assignedTeacher = null;
}

```

```
/**
```

```
* getCourseName() returns course name
```

```
* setCourseName(String name) sets course name
```

```
* getSkillsNeeded() returns required skills
```

```
* setSkillsNeeded(String skills) sets required skills
```

```

* getHours() returns hours
* setHours(int hours) sets hours
* getAssignedTeacher() returns the assigned teacher (or null)
* setAssignedTeacher(Teacher teacher) assigns a teacher
* getId() returns the requirement's id
* toString() returns a readable summary
*/
public int getId() {return id;}
public String getCourseName() {return courseName;}
public void setCourseName(String name) {this.courseName = name;}
public String getSkillsNeeded() {return skillsNeeded;}
public void setSkillsNeeded(String skills) {this.skillsNeeded = skills;}
public int getHours() {return hours;}
public void setHours(int hours) {this.hours = hours;}
public Teacher getAssignedTeacher() {return assignedTeacher;}
public void setAssignedTeacher(Teacher teacher) {this.assignedTeacher = teacher;}

@Override
public String toString(){
    String teacherInfo = (assignedTeacher ==null)
        ? "None"
        : assignedTeacher.getName();

    return String.format("[Req %d] %s | Skills: %s | Hours: %d | Teacher: %s",
        id, courseName, skillsNeeded, hours, teacherInfo);
}

}

```

6. CODE: ENTITYSTORE--> FileHandler.Java

```
package Entitystore;
```

```
import java.io.*; import java.util.ArrayList; import java.util.List; import Entities.Teacher; import
Entities.TeachingRequirement;
```

```
/**
```

- Handles loading and saving of teacher and teaching requirement data to CSV files.
-
- File formats:
- -
 - Teachers CSV: {@code id,name,skills,trainingStatus}
 -
 - Requirements CSV: {@code id,courseName,skillsNeeded,hours,assignedTeacherId}
 -
- The {@code assignedTeacherId} field is {@code -1} if no teacher is assigned.

```

*/ public class FileHandler {

// ===== LOADING METHODS =====

/**
 * Reads teachers from a CSV file and returns them as a list.
 * The file format is expected to be: {@code id,name,skills,trainingStatus}.
 * Lines starting with '#' are treated as comments and ignored.
 * If the file does not exist, an empty list is returned and a message is printed.
 *
 * @param filepath the path to the teachers CSV file
 * @return a list of Teacher objects loaded from the file, or an empty list if the file is missing or unreadable
 */
public static List<Teacher> loadTeachers(String filepath) {
    List<Teacher> teachers = new ArrayList<>();
    File file = new File(filepath);

    //if the file doesn't exist yet, return empty list
    if (!file.exists()) {        System.out.println("No existing teacher data found. Starting fresh.");
        return teachers;
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) continue;
            String[] parts = line.split(",", 4);
            if (parts.length == 4) {
                int id = Integer.parseInt(parts[0].trim());
                String name = parts[1].trim();
                String skills = parts[2].trim();
                String trainingStatus = parts[3].trim();
                //use of the constructor that takes an existing id
                Teacher teacher = new Teacher(id, name, skills, trainingStatus);

                teachers.add(teacher);
            }
        }
        System.out.println("Loaded teachers from file.");
    } catch (IOException e) {
        System.out.println("Error reading teachers file: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.out.println("Error parsing teacher data: " + e.getMessage());
    }

    return teachers;
}

/**

```

- * Reads teaching requirements from a CSV file and returns them as a list.
- * The file format is expected to be: {@code id,courseName,skillsNeeded,hours,assignedTeacherId}.
- * Lines starting with '#' are treated as comments and ignored.
- * If a valid teacher ID is present, the corresponding Teacher object is linked using the provided teacher list.
- * If the file does not exist, an empty list is returned and a message is printed.
- *
- * @param filepath the path to the requirements CSV file
- * @param teachers the list of all teachers (used to resolve assigned teacher references)
- * @return a list of TeachingRequirement objects loaded from the file, or an empty list if the file is missing or unreadable
- */

```
public List<TeachingRequirement> loadRequirements(String filepath, List<Teacher> teachers) {
    List<TeachingRequirement> requirements = new ArrayList<>();
    File file = new File(filepath);

    //if the file doesn't exist yet, return empty list
    if (!file.exists()) {
        System.out.println("No existing requirements data found. Starting fresh.");
        return requirements;
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) continue;
            String[] parts = line.split(",", 5);
            if (parts.length == 5) {
                int id = Integer.parseInt(parts[0].trim());
                String courseName = parts[1].trim();
                String skillsNeeded = parts[2].trim();
                int hours = Integer.parseInt(parts[3].trim());
                int assignedTeacherId = Integer.parseInt(parts[4].trim());

                //use of the constructor that takes an existing id
                TeachingRequirement req = new TeachingRequirement(id, courseName, skillsNeeded, hours);

                //link of the assigned teacher if there is one
                if (assignedTeacherId != -1) {
                    Teacher assignedTeacher = findTeacherById(teachers, assignedTeacherId);
                    if (assignedTeacher != null) {
                        req.setAssignedTeacher(assignedTeacher);
                    }
                }

                requirements.add(req);
            }
        }
    }
    System.out.println("Loaded requirements from file.");
} catch (IOException e) {
    System.out.println("Error reading requirements file: " + e.getMessage());
}
```

```

    } catch (NumberFormatException e) {
        System.out.println("Error parsing requirement data: " + e.getMessage());
    }

    return requirements;
}

// ===== SAVING METHODS =====

/**
 * Writes all teachers to a CSV file.
 * The file format is: {@code id,name,skills,trainingStatus}.
 * A header comment line is written first for readability.
 *
 * @param filepath the path to the teachers CSV file
 * @param teachers the list of Teacher objects to save
 */
public static void saveTeachers(String filepath, List<Teacher> teachers) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filepath))) {
        //write a header comment so the file is human-readable
        writer.write("# id,name,skills,trainingStatus");
        writer.newLine();

        for (Teacher teacher : teachers) {
            String line = teacher.getId() + ","
                + teacher.getName() + ","
                + teacher.getSkills() + ","
                + teacher.getTrainingStatus();
            writer.write(line);
            writer.newLine();
        }
        System.out.println("Saved " + teachers.size() + " teachers to file.");
    } catch (IOException e) {
        System.out.println("Error saving teachers file: " + e.getMessage());
    }
}

/**
 * Writes all teaching requirements to a CSV file.
 * The file format is: {@code id,courseName,skillsNeeded,hours,assignedTeacherId}.
 * A header comment line is written first for readability.
 * If a teacher is assigned, the teacher's ID is stored; otherwise, {@code -1} is stored.
 *
 * @param filepath the path to the requirements CSV file
 * @param requirements the list of TeachingRequirement objects to save
 */
public void saveRequirements(String filepath, List<TeachingRequirement> requirements) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filepath))) {
        writer.write("# id,courseName,skillsNeeded,hours,assignedTeacherId");
        writer.newLine();
    }
}

```

```

for (TeachingRequirement req : requirements) {
    // -1 if no teacher is assigned
    int teacherId = (req.getAssignedTeacher() != null)
        ? req.getAssignedTeacher().getId()
        : -1;

    String line = req.getId() + ","
        + req.getCourseName() + ","
        + req.getSkillsNeeded() + ","
        + req.getHours() + ","
        + teacherId;
    writer.write(line);
    writer.newLine();
}
System.out.println("Saved " + requirements.size() + " requirements to file.");
} catch (IOException e) {
    System.out.println("Error saving requirements file: " + e.getMessage());
}
}

// ===== HELPER METHODS =====

/**
 * Finds a teacher by its ID from a list of teachers.
 *
 * @param teachers the list of Teacher objects to search
 * @param id the ID to look for
 * @return the Teacher with the matching ID, or {@code null} if none found
 */
private Teacher findTeacherById(List<Teacher> teachers, int id) {
    for (Teacher teacher : teachers) {
        if (teacher.getId() == id) {
            return teacher;
        }
    }
    return null;
}
}

```

C) DATA

Data used in the system requirements.csv

1	# id	courseName	skillsNeeded	hours	assignedTeacherId
2	1	Intro to Programming	Java Programming	20	-1
3	2	Database Fundamentals	Database Systems	15	2
4	3	Internet Technology	Web Development	25	-1
5	4	Software Quality Assurance	Software Testing	10	-1
6	5	Cybersecurity Fundamentals	Cryptography	9	6
7	6	Programming	Python	8	6
8	7	Maths	Calculus	19	7

Data teachers.csv

1	# id	name	skills	trainingStatus
2	1	John Smith	Java Programming	Completed
3	2	Sarah Wilson	Database Systems	In Progress
4	3	David Brown	Web Development	Not Started
5	4	Emily Clark	Software Testing	Completed
6	5	James Taylor	Machine Learning	In Progress
7	6	Luke Roche	Python	Completed
8	7	Joe Blogs	Calculus	Completed